LAM Research Challenge 2025

Stage-1: Logical League

Team LAMe: LRC-U-0517-1

Members:

Adithya Vishnu

Harsh Chandwani

Sreeja Srinivasan

Sparsh Pradeep Jain

Key challenges handled

- **Obstacle Configuration**

To evaluate the system's obstacle detection and navigation capabilities, **three distinct obstacles** were strategically positioned within the arena to introduce varying levels of difficulty.

- **Obstacle 1:** Placed along a **straight path**, designed to test basic obstacle detection and removal efficiency.

- **Obstacle 2:** Located **on a corner**, introducing a moderate challenge due to limited maneuvering space.

- **Obstacle 3:** Positioned **immediately after a turn**, simulating a complex real-world obstruction scenario that required precise timing and coordination between the SRM and ALFR.

This configuration ensured a progressive difficulty setup, allowing systematic testing of both detection accuracy and response timing during obstacle clearance.

- **Obstacle Interaction Logic**

The obstacle interaction mechanism involved two coordinated systems — the **Support Robot Module (SRM)** and the **Autonomous Line-Following Robot (ALFR)**.

- The **SRM** was responsible for **picking up**, and **relocating** obstacles away from the line path to maintain a clear route for the ALFR.

- The **ALFR** was programmed to **stop automatically** upon detecting an obstacle and **resume navigation** only after the SRM had successfully cleared it.

During testing, **three different obstacle shapes —** a **cube**, a **cylinder**, and a **sphere —** were used. The SRM's gripper arm was able successfully lift each shape, demonstrating versatility and precision in obstacle handling.

- **Time Limit**

The arena challenge included a time-based performance metric, requiring the task to be completed within **10 minutes**.

- In the **baseline test** (without obstacles), the ALFR successfully navigated the entire arena in **40 seconds**, showcasing efficient line-following and path optimization.

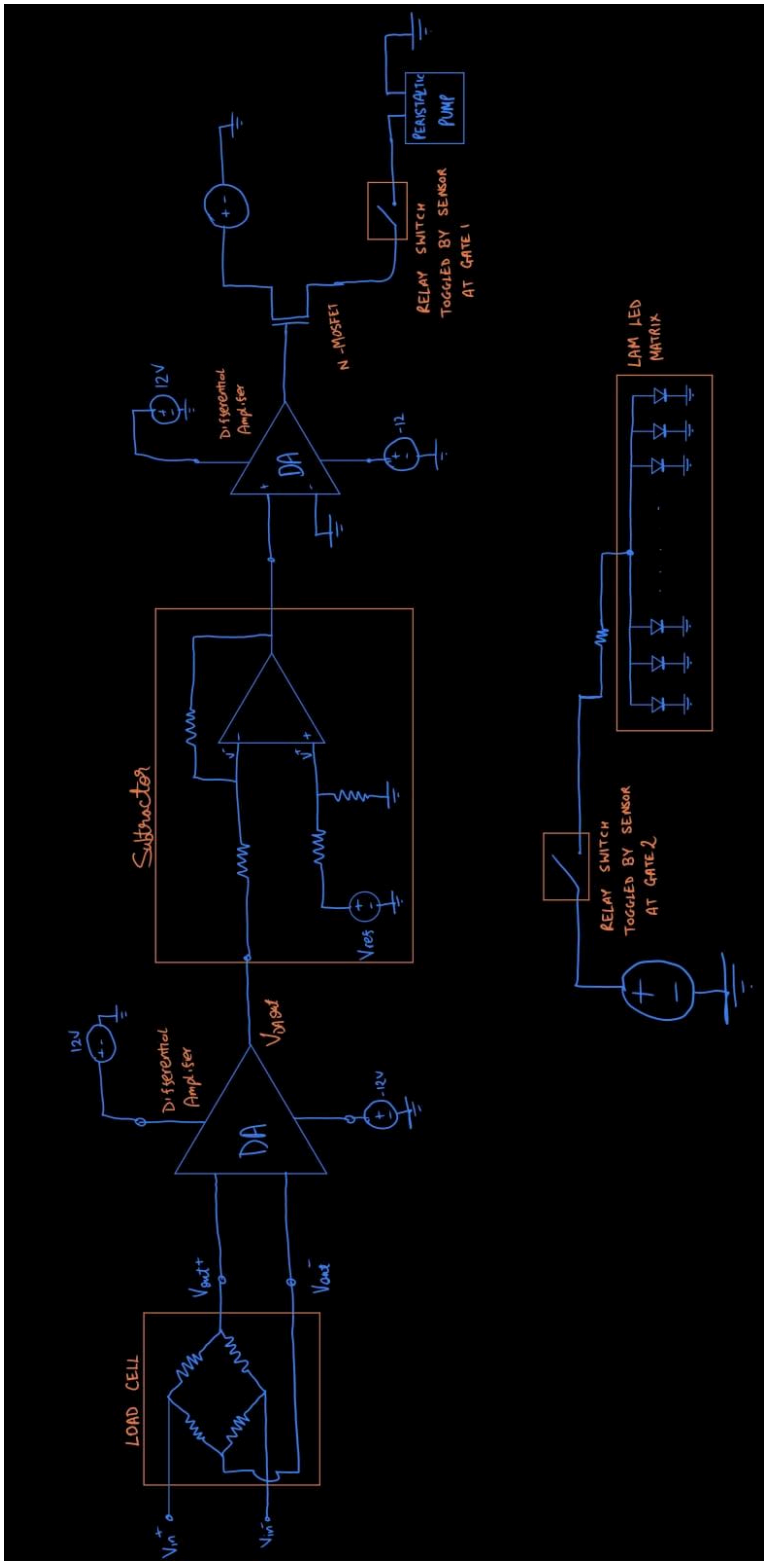
- In the **full test** (with all obstacles present), the combined system of SRM and ALFR completed the entire course in **3 minutes**, clearing each obstacle sequentially as encountered.

This performance demonstrated reliable coordination between the robots and efficient task execution well within the prescribed time limit.

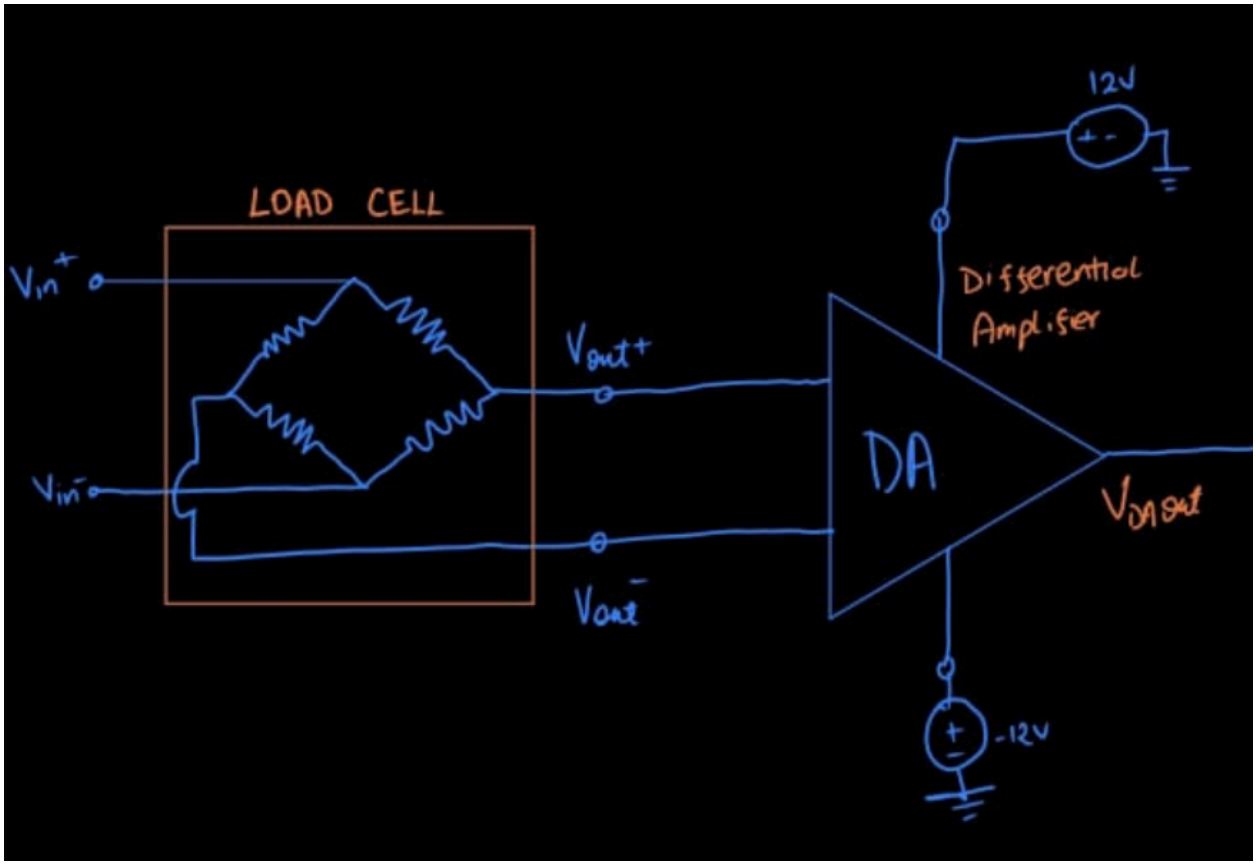
- **Peristaltic Pump Verification**

We have used a load cell to measure the weight of water dispensed and all other components of the circuits will be tuned such that it triggers the switch at the exact weight required.

Circuit Diagram:



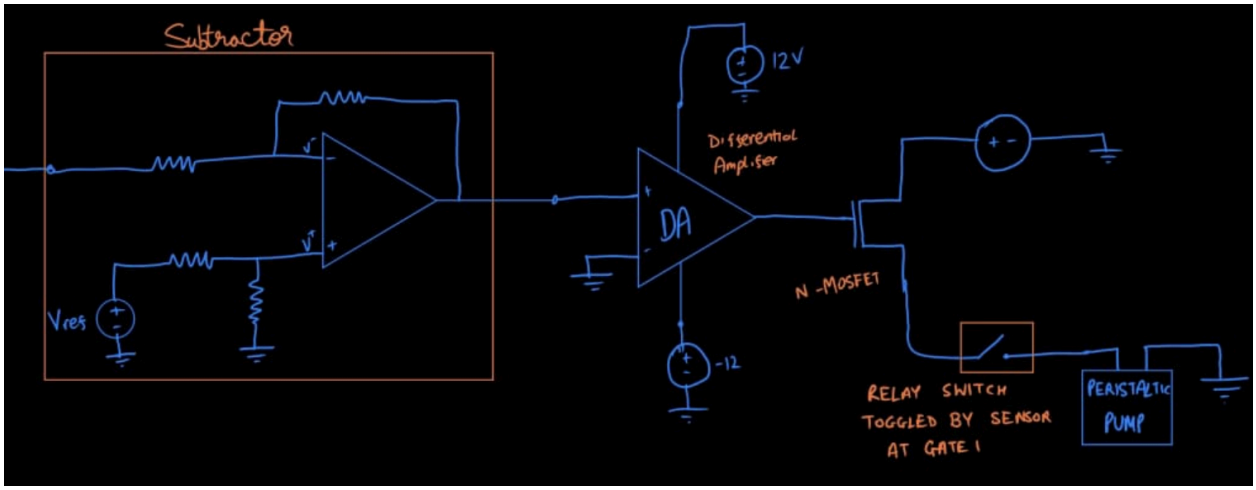
Detailed Explanation:



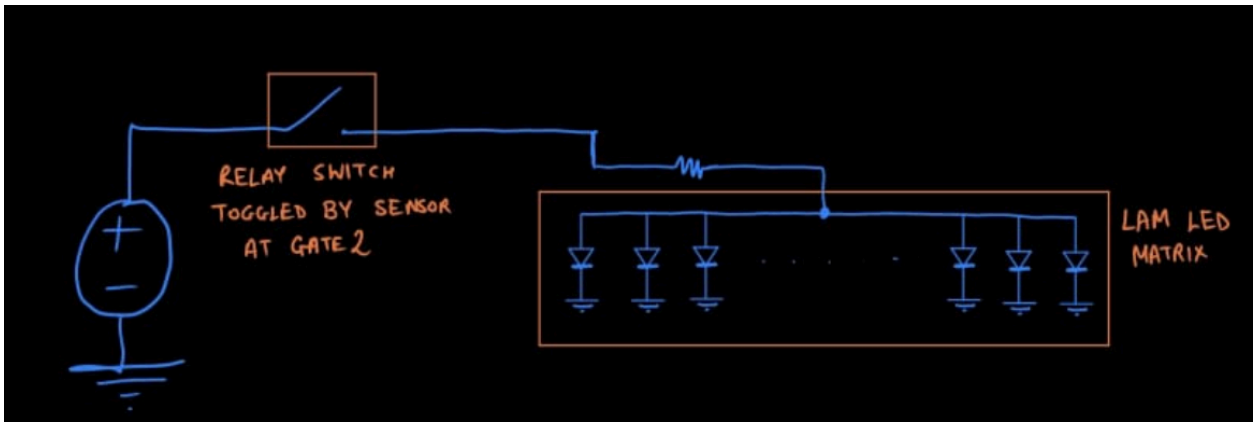
A load cell is a transducer that converts mechanical force (weight or load) into an electrical signal. It operates based on the strain gauge principle, where a change in force causes a corresponding change in electrical resistance.

When weight is applied to the load cell, the metal body (elastic element) deforms slightly. This deformation alters the resistance of the strain gauges attached to it, which are connected in a Wheatstone bridge configuration. The resulting output is a small differential voltage proportional to the applied load.

Since this voltage is typically in the millivolt range (mV), it must be amplified using an instrumentation amplifier such as the HX711 module.



An Op-Amp has been used to implement a subtractor which measures the difference between load cell output and a reference voltage which is then used to control a N-MOSFET switch.



On passing gate 2 a relay switch is triggered which in turn switches on the LAM LED Matrix

Key features of custom code for line following of ALFR:

1. Error Calculation with 5 IR sensors:

The five IR sensors return intensity values (black < 0.5).

Each sensor is assigned a weight (−4000 to +4000) based on its position.

The error is the weighted average of all active (black-detecting) sensors:

```
local position = 0
local activeCount = 0
for i = 1, 5 do
    if sensorValues[i] < 0.5 then
        position = position + ((i - 3) * 2000)
        activeCount = activeCount + 1
    end
end
```

2. PID Control:

Computes proportional, integral, and derivative terms of the error to correct steering smoothly.

The PID parameters were carefully tuned to achieve maximum speed and accuracy in line tracking.

```
local P = error
I = math.max(math.min(I + error, 1000), -1000)
local D = error - lastError

local correction = Kp * P + Ki * I + Kd * D

leftV = baseSpeed - (correction / 400) * 3
rightV = baseSpeed + (correction / 400) * 3
```

3. Obstacle Handling:

Stops the robot when the proximity sensor detects an obstacle.

```
local detectionState, distance, detectedPoint, detectedObjectHandle = sim.readProximitySensor(sensingNose)
```

```
if detectionState > 0 then
    sim.setJointTargetVelocity(leftMotor, 0)
    sim.setJointTargetVelocity(rightMotor, 0)
```

4. Line Recovery:

If all sensors lose the line, the robot turns in the direction of the last known error until the line is found again.

```
if activeCount > 0 then
    error = position / activeCount
else
    -- Line lost ? recovery
    if lastError > 0 then
        leftV = -0.2
        rightV = 0.2
    else
        leftV = 0.2
        rightV = -0.2
    end
```

Key features of custom code for manual control of Single Arm Robot:

1. Custom Graphical UI:

Creates an on-screen control panel using simUI.create() with buttons for forward, backward, left, right, rotation, stop, and pick operations.

```
xml = [[
<ui title="YouBot Controller" closeable="true" resizable="false" activate="false">
    <button text="Backward" on-click="moveForward"/>
    <button text="Forward" on-click="moveBackward"/>
    <button text="Left" on-click="moveLeft"/>
    <button text="Right" on-click="moveRight"/>
    <button text="Rotate Left" on-click="rotateLeft"/>
    <button text="Rotate Right" on-click="rotateRight"/>
    <button text="Stop" on-click="stop"/>
    <button text="Pick Object" on-click="pickObject"/>
</ui>
]]
```

2. Mecanum Wheel Drive System:

Uses mecanum wheels to achieve omnidirectional movement, allowing the robot to move forward, backward, sideways, and rotate in place.

```
function setMovement(forwBackVel, leftRightVel, rotVel)
    sim.setJointTargetVelocity(wheelJoints[1], -forwBackVel - leftRightVel - rotVel)
    sim.setJointTargetVelocity(wheelJoints[2], -forwBackVel + leftRightVel - rotVel)
    sim.setJointTargetVelocity(wheelJoints[3], -forwBackVel - leftRightVel + rotVel)
    sim.setJointTargetVelocity(wheelJoints[4], -forwBackVel + leftRightVel + rotVel)
end
```

3. Arm and Gripper Control:

Functions to position the robotic arm and open/close the gripper using joint target positions.

```
function setArmPositions(p)
    for i = 1, #armJoints do
        sim.setJointTargetPosition(armJoints[i], p[i])
    end
end

function setGripper(open)
    local pos = open and 0.025 or 0.0
    sim.setJointTargetPosition(gripperJoints[1], pos)
    sim.setJointTargetPosition(gripperJoints[2], -pos)
end
```

4. Pick-and-Place Operation:

Automated routine configured to pick an object: move to a ready pose, lower arm, close gripper, attach object, and lift it.

```
function pickObject()
    sim.addLog(sim.verbosity_scriptinfos, "Starting pick operation...")

    stop()

    -- Step 1: Move to ready pose
    setArmPositions({0, 0.4, -1.0, 1.6, 0})
    setGripper(true) -- open
    sim.wait(2)

    -- Step 2: Move arm down to object
    setArmPositions({0, 0.6, -1.3, 1.4, 0})
    sim.wait(2)

    -- Step 3: Close gripper to grab
    setGripper(false)
    sim.wait(1)

    -- Optional: virtually attach object (for visualization)
    if cuboid >= 0 then
        sim.setObjectParent(cuboid, gripperJoints[1], true)
    end

    -- Step 4: Lift object
    setArmPositions({0, 0.3, -0.9, 1.4, 0})
    sim.wait(2)

    sim.addLog(sim.verbosity_scriptinfos, "Pick operation complete.")
end
```

Key features of code for LAM LED turning on when ALFR passes through gate 2:

1. LED color control

Turns LEDs red when activated, and black otherwise.

```
for i, led in ipairs(leds) do
    if ledsOn then
        sim.setShapeColor(led, nil, sim.colorcomponent_ambient_diffuse, {1, 0, 0})  -- Red ON
    else
        sim.setShapeColor(led, nil, sim.colorcomponent_ambient_diffuse, {0, 0, 0})  -- Off
    end
end
```

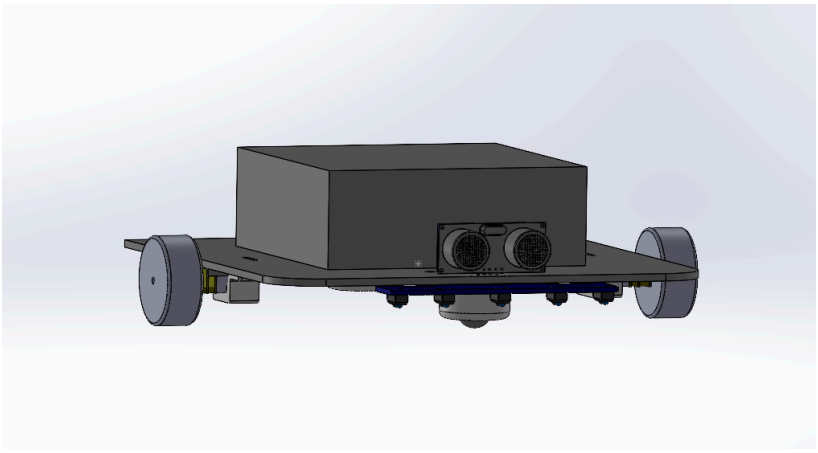
2. Persistent state flag

Keeps LEDs on once triggered, even after the object moves away.

```
if result > 0 then
    ledsOn = true
end
```

CUSTOM MODELS

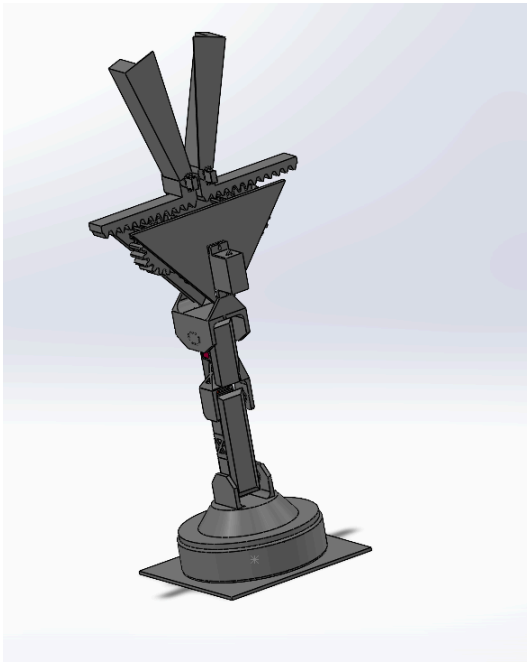
ALFR:



So coming to the model of Automatic Line follower Robot, we chose to go with this design because it has rotational symmetry, less moment of inertia to reduce chances of overshooting, which is really important for most efficient path following and preventing repetitive errors. And to remove any chance of rotation asymmetry we have provided two caster wheels at the front and the back, to remove any chance of uneven torque distribution.

Other than that we have made the design such that there is no chance of toppling and the mass is centred since the battery and other electronic components are kept at the centre.

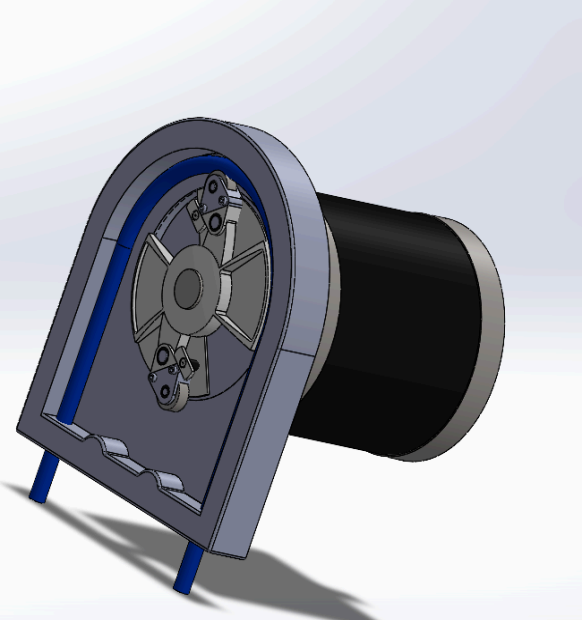
SINGLE ARM - ROBOT



So the bot consists of multiple links. It starts from a stationary base that is attached to the mechanum wheel base. Then the rotating body is attached onto this stationary base through a servo and ball bearing for additional support. Now the first extension and second extension are designed to be of minimal weight and have enough supports (additional triangulations have been provided for this purpose to reduce the material used.)

Now coming to the gripper, we have made the gripper such that it is adjustable for varying sizes and shapes of objects. This uses the principle of rack and pinion gears and a set of three intersecting spur gears to transfer the torque from one servo motor to both the set of racks equally thereby expanding the distance between the two arms. Then the design includes two additional servos on each rack to better adjust the arms to fit regardless of what shape the object is of.

PERISTALTIC PUMP



Coming to the basic design of the peristaltic pump, We have used two rollers connected to a motor, that revolves the rollers periodically and controlled, which pushes specific volumes of fluid through the tube that can be controlled by controlling the number of rotations and changing the thickness of the tube.